

Family App — Technical Architecture

This is an as-built technical specification of the app as it currently stands. Unlike `family-app-project-brief.md` (the original pre-build wishlist — now partly outdated: budget was cut, magic-link login was replaced, meal planning was added), this document describes what actually exists in the codebase today.

1. Tech Stack

Layer	Technology
Framework	Expo SDK 57 (React Native 0.86, React 19.2.3)
Language	TypeScript (strict mode)
Navigation	<code>expo-router</code> (file-based routing)
Backend	Supabase (Postgres, Auth, Realtime, Edge Functions)
Auth	Supabase anonymous auth — no email/password/magic link
State/data	Plain React hooks per data domain, no Redux/Zustand/React Query
Push notifications	<code>expo-notifications</code> + Expo Push API, fanned out via a Supabase Edge Function
Localization	<code>i18next</code> / <code>react-i18next</code> , English + French, in-app toggle
Styling	Hand-written <code>StyleSheet.create</code> + a shared <code>lib/theme.ts</code> token file (no styling library)
Build/distribution	EAS Build (Android APK via <code>preview</code> profile); iOS not yet distributed (see below)

Key dependencies (from `package.json`): `@supabase/supabase-js`, `expo-router`, `expo-notifications`, `expo-localization`, `react-native-gesture-handler`, `@react-native-community/datetimepicker`, `@react-native-async-storage/async-storage`, `expo-linear-gradient`.

2. Project Structure

app/	Screens – one file per route (expo-router)
_layout.tsx	Root layout: gates rendering on i18n init, wraps AuthPro
(app)/	
_layout.tsx	Auth/session gate + family-membership gate (FamilyGate)
(tabs)/	
_layout.tsx	Bottom tab bar (Home, Calendar, To-Do, Groceries, Meals)
index.tsx	Home tab
calendar.tsx	Calendar tab
todos.tsx	To-Do tab
groceries.tsx	Groceries tab
meals.tsx	Meals tab (weekly meal plan)
event/[id].tsx	Event detail/edit
todo/[id].tsx	To-do detail/edit
grocery/[id].tsx	Grocery item detail/edit
recipe/[id].tsx	Recipe detail/edit (id === "new" for creation)
recipes.tsx	Recipe library/browser
timetable.tsx	Per-member weekly work/school timetable
calendar-settings.tsx	Holiday display + member colors
todo-settings.tsx	Member colors (shared with calendar)
settings.tsx	Profile, family, members, language, home sections, no
setup.tsx	First-launch create/join family flow
join-family.tsx	Join an additional family from Settings
components/	Shared, mostly presentational UI components
calendar/	Calendar-view-specific pieces (MonthGrid, WeekHourGrid,
Button.tsx, Chip.tsx, TextField.tsx, FieldLabel.tsx, ModalTitle.tsx,	
BottomSheetModal.tsx, TabScreenHeader.tsx, ColorSwatchPicker.tsx,	
RecurrencePicker.tsx, EventReminderPicker.tsx, TodoReminderPicker.tsx,	
TodoCategoryPicker.tsx, RecipeForm.tsx, RecipeListView.tsx,	
IngredientListEditor.tsx, AddIngredientsToGroceriesModal.tsx	
ErrorBoundary.tsx / ErrorScreen.tsx	Global crash fallback (intentionally English-
hooks/	Data layer – one hook per domain, talks directly to Supa
useAuth.tsx	Anonymous session bootstrap (Context)
useProfile.tsx	Current user's profile row (Context)
useFamily.ts	Active family (name, invite code)
useMyFamilies.ts	All families the user has joined + switching between t
useFamilyMembers.ts	Member list, roles, colors, admin actions (promote/rem
useEvents.ts	Calendar events + participants
useCalendarPrefs.ts	Per-viewer holiday display prefs
useTodos.ts	To-dos

<code>useTodoCategories.ts</code>	Custom to-do categories
<code>useGroceries.ts</code>	Grocery items + places ("categories")
<code>useMealPlan.ts</code>	Weekly meal plan entries
<code>useRecipes.ts</code>	Recipes + ingredients
<code>useTimetable.ts</code>	Per-member weekly schedule blocks + date overrides
<code>useMessages.ts</code>	Home's one-slot-per-member "important message"
<code>usePushNotifications.ts</code>	Registers/refreshes this device's Expo push token
<code>useReminderScheduler.ts</code>	Schedules *local* on-device notifications for persona
<code>usePinchZoom.ts, useSwipeNavigate.ts</code>	Generic gesture helpers, no Supabase involve
<code>lib/</code>	Framework-agnostic utilities, no React
<code>supabase.ts</code>	Supabase client singleton
<code>theme.ts</code>	Colors, spacing, gradients, radii
<code>dateUtils.ts</code>	Date math + locale-aware formatDate/formatTime (follow
<code>recurrence.ts</code>	Recurrence rule → concrete occurrence expansion (calen
<code>reminders.ts</code>	Event/to-do reminder label + summary formatting
<code>timetable.ts</code>	Time-string helpers + weekly occurrence expansion for
<code>memberColors.ts</code>	Per-member color resolution (custom or hash-based fal
<code>newBadge.ts</code>	"Is this item new to me?" check, drives all "New" badg
<code>holidayMarkers.ts / frenchHolidays.ts</code>	French public + school holiday data and ran
<code>recipeCategories.ts, mealTypes.ts, todoCategoryIcons.ts, groceryPlaces.ts</code>	Small p
<code>globalErrorHandler.ts</code>	Hooks into RN's global error handler for the crash scr
<code>i18n/</code>	<code>index.ts</code> (init/setup), <code>en.ts</code> , <code>fr.ts</code> (translation resou
<code>supabase/</code>	
<code>migrations/</code>	31 sequential SQL migrations – the database schema's f
<code>functions/send-notification/</code>	Edge Function that sends push notifications
<code>eas.json</code>	EAS Build profiles (Android preview build config, incl.
<code>app.json</code>	Expo app config (bundle IDs, plugins, icons)

3. Frontend Architecture

Routing

File-based via `expo-router`. `app/_layout.tsx` is the true root: it initializes `i18n` (async, shows a spinner until ready — see §7), wraps everything in `GestureHandlerRootView` and a global `ErrorBoundary`, and provides `AuthProvider`. `app/(app)/_layout.tsx` sits inside that and does two sequential gate checks before rendering any screen:

1. **AppLayout**: waits on `useAuth()`; if there's no session at all (offline on first launch), shows a plain "couldn't connect" message instead of the app.
2. **FamilyGate**: waits on `useProfile()`; if the user has no `family_id` yet, redirects to `/setup`; if they do have one and are still on `/setup`, redirects away from it. This is also where `usePushNotifications()` and `useReminderScheduler()` are mounted — so both start running as soon as a session + profile exist, for every screen under this layout.

The hook-per-domain pattern

Every feature has one hook (e.g. `useTodos`, `useEvents`, `useGroceries`) that is the *only* thing screens talk to — no screen calls `supabase` directly for reads. Every one of these hooks follows the same shape:

1. **Local state**: `const [items, setItems] = useState<T[]>([])`.
2. **refetch()**: an async function that runs the actual `select` query/queries and replaces local state wholesale. Scoped implicitly by RLS (see §4) — most queries don't even need an explicit `family_id` filter client-side, since Postgres only returns rows the caller is allowed to see.
3. **Initial fetch + Realtime subscription** (in a `useEffect`): calls `refetch()` on mount, then opens a Supabase Realtime channel subscribed to `postgres_changes` on the relevant table(s), filtered by `family_id` where applicable. Any INSERT/UPDATE/DELETE from *any* device triggers another `refetch()`. **Channel names always include `useId()`** (e.g. ``todos:${familyId}:${instanceId}``) — a per-component-instance id — because two mounted instances subscribing to the same literal channel name crashes natively in a way the JS error boundary never sees.
4. **Mutations** (`addX`, `updateX`, `deleteX`, `toggleX`): call `supabase.from(...).insert/update/delete(...)`. Destructive/toggle actions apply an **optimistic local update first**, then fire the network call, and roll back via `refetch()` only if it errors. Additive actions (`addTodo`, `addItem`, `addRecipe`) use `.select().single()` on the insert so the new row can be pushed straight into local state immediately, rather than waiting for the Realtime event to arrive (which is not instantaneous and occasionally drops — this exact gap in `addTodo` was a real bug fixed in this codebase's history).
5. Screens with modal-based "add" flows that don't navigate away also add a `useFocusEffect(() => refetch())` as a belt-and-suspenders refresh, since the Realtime channel is the primary sync mechanism but not treated as 100% reliable on its own.

Family membership vs. active family

`profiles.family_id` is the user's *currently active* family — but a user can belong to several (`family_members` join table, `useMyFamilies`/`switchFamily`). Anything that needs "everyone who should ever see/be notified about this," like the push notification Edge Function, deliberately queries `family_members` rather than `profiles.family_id`, so someone currently browsing a different family still gets notified about the one they're not looking at.

Theming, colors, badges

- `lib/theme.ts` is the single source of design tokens (colors, spacing, gradients); every screen imports from it rather than hardcoding values.
- Member colors are **per-viewer**, not global: each person can assign their own color to each other member (`member_color_prefs` table), falling back to a deterministic hash of the member's id (`lib/memberColors.ts`) if they haven't picked one. "Whole family" / "unassigned" always render in a fixed neutral black, never a member color.
- The "New" badge shown on rows across Calendar/To-Do/Groceries is driven by one shared rule (`lib/newBadge.ts`): an item is "new" if it was created after `profile.last_seen_activity_at` and wasn't created by the viewer themselves. Clearing Home's "What's New" section bumps that timestamp and clears every badge everywhere at once.

4. Backend Architecture (Supabase)

Auth

Purely **anonymous auth** (`supabase.auth.signInAnonymously()`), no login screen at all. On first launch, every device silently gets a brand-new anonymous session and goes straight to the create/join-family flow (`setup.tsx`). There is no email or password, so identity is tied to the device/app install — losing it means losing that identity (this is an explicit, documented design choice, not an oversight). A Postgres trigger (`handle_new_user`, fires on `auth.users` insert) auto-creates a matching blank `profiles` row.

"Managed members" (added via Settings, for people without a phone/login of their own — e.g. young kids) are the one exception: they get a `profiles` row with a freshly generated id and **no** corresponding `auth.users` row, which is why the FK from `profiles.id → auth.users.id` had to be dropped from the schema.

Database schema (current shape — 31 migrations' worth of evolution)

Family/auth:

- **families** — `id, name, invite_code` (unique 6-char code), `created_at`.
- **profiles** — one per person (real or managed): `id, family_id` (active family), `full_name, is_managed, managed_by, last_seen_activity_at, home_visible_sections[], notification_prefs` (jsonb), `created_at`.
- **family_members** — many-to-many join enabling multi-family membership: `profile_id, family_id, role` ('admin'|'user'), `joined_at`.

Calendar:

- **events** — `title, description, location, start_at, all_day, applies_to_whole_family, is_private, created_by, event_type`, plus recurrence columns (`recurrence_freq/interval/days_of_week/end_type/end_date/count`) and `reminder_offsets_minutes int[]`.
- **event_participants** — `event_id, profile_id` join, used when an event isn't for the whole family.
- **calendar_prefs** — per-viewer-per-family holiday display settings (`holiday_color, school_zone, show_public_holidays, show_school_holidays`).
- **member_color_prefs** — `viewer_id, member_id, color` — one row per (viewer, member) pair.

To-do:

- **todos** — `title, is_complete, assigned_to, priority` ('urgent'|'soon'|'whenever'), `description, is_private, category_id, due_date, reminder_enabled/freq/time/weekday, created_by, completed_at`.
- **todo_categories** — custom family-wide labels (`name, icon, created_by`), orthogonal to the hardcoded priority levels. Note: currently only has SELECT/INSERT RLS policies — no rename/delete supported server-side yet.

Groceries:

- **grocery_categories** — "places" (e.g. a store); `name, is_default` (every family gets one permanent default place, "Anywhere," flagged via `is_default` rather than matched by name so it can be freely renamed).
- **grocery_items** — `name, category_id, is_checked, is_archived` (kept as history for "recently used" suggestions), `description, created_by, checked_at`. Has a DB-level unique index on `(family_id, category_id, lower(name))` where not `is_archived` to guard against duplicate concurrent adds from two devices.

- `grocery_saved_lists` — one reusable item-name template per place (`item_names text[]`) — not currently wired into Realtime.

Meals/recipes:

- `recipes` — `name, details, categories text[]` (checked against a fixed set: `appetizer/main/side/dessert/snack/breakfast/drink`), `created_by`.
- `recipe_ingredients` — `recipe_id, quantity, name, sort_order`.
- `meal_plan_entries` — `date, meal_type ('breakfast'|'lunch'|'snack'|'dinner')`, `recipe_id` (nullable — quick-menu entries skip it), `title, serves, details, created_by`.

Timetable:

- `timetable_blocks` — recurring weekly schedule per member: `profile_id, day_of_week` (`0=Mon..6=Sun`), `start_time, end_time, label`.
- `timetable_overrides` — per-date exceptions: `block_id, override_date, is_cancelled, start_time, end_time, label`.

Messaging & push:

- `messages` — one row per member (`profile_id` is the PK), `content` (`≤30 chars`), `updated_at` — a single "pinned message" slot, overwritten on repost.
- `push_tokens` — `profile_id, token (unique)` — this device's Expo push token.

Row Level Security

Nearly every table is scoped by `family_id = public.get_my_family_id()`, a `SECURITY DEFINER` SQL helper that looks up the caller's active family from `profiles`. This helper exists specifically to dodge RLS self-recursion — a plain subquery against `profiles` inside a `profiles`-touching policy re-triggers RLS on itself. `families`/`family_members` use a broader variant, `get_my_family_ids()` (returns *every* family the user has joined), since those tables need to stay visible across all memberships, not just the active one.

Notable deviations from the plain family-scoped pattern:

- **Private items:** `events` and `todos` add `and (is_private = false or created_by = auth.uid())` to their SELECT policy — invisible to the rest of the family, visible only to the creator, even within the same family.

- **Messages:** any family member can SELECT, but only the owning `profile_id` can INSERT/UPDATE/DELETE their own row — admins additionally get a delete policy (Postgres OR-combines multiple permissive policies on the same command).
- **Sensitive cross-member actions** (recoloring, promoting, removing, renaming another member, switching active family) are deliberately **not** exposed as loose UPDATE policies on `profiles`/`family_members` — they only happen through the RPCs below, keeping the base table policies self-only.

RPCs (SECURITY DEFINER functions)

All callable via `supabase.rpc(...)`, all `set search_path = public`, granted to `authenticated` only:

RPC	Purpose
<code>create_family(family_name, member_name)</code>	Creates a family + invite code, makes the caller its first <code>admin</code> , seeds the default "Anywhere" grocery place, sets the caller's profile
<code>join_family(code, member_name)</code>	Looks up family by invite code, adds caller as <code>role='user'</code> , sets active family
<code>switch_active_family(target_family_id)</code>	Repoints <code>profiles.family_id</code> to another family the caller already belongs to
<code>promote_to_admin(target_profile_id, target_family_id)</code>	Admin-only: upgrades another member's role
<code>remove_family_member(target_profile_id, target_family_id)</code>	Admin-only: removes membership, cleans up their pinned message, and — if the target is a managed member — deletes their whole profile outright
<code>add_managed_member(target_family_id, member_name)</code>	Any member can create a phone-less "managed" profile (e.g. for a child)
<code>rename_member(target_member_id, new_name)</code>	Any member sharing a family with the target can rename them
<code>get_my_family_id()</code> / <code>get_my_family_ids()</code>	Internal RLS helpers, not meant to be called from the client for business logic

RPC	Purpose
<code>notify_activity()</code>	Trigger function (not client-callable) — see §5

Realtime

Standard `supabase_realtime` publication, with every syncable table added individually (`grocery_saved_lists` and `push_tokens` are the two intentional exceptions — never need cross-device sync). Every table also sets `replica identity full` — required for filtered subscriptions (`family_id=eq.X`) to work correctly on DELETE, since without full replica identity Postgres's logical replication only ships the primary key of a deleted row, which the client-side filter can't match against.

Edge Function: `send-notification`

The only Edge Function in the project (`supabase/functions/send-notification/index.ts`, Deno). Fired by a lightweight `pg_net`-based Postgres trigger (`notify_activity()`, wired to `events`, `todos`, `grocery_items` on INSERT, and `messages` on INSERT-or-UPDATE) that POSTs `{ type, table, record }` to the function's URL. The function:

1. Maps the table to a notification category (`events→calendar`, `todos→urgentTodos | todos` based on priority, `grocery_items→groceries`, `messages→messages`).
2. Skips silently if the row is private (defense-in-depth on top of RLS).
3. Looks up the author's name, resolves every *other* member of the family via `family_members` (not `profiles.family_id`, for the reason noted in §3), filters them by their own `notification_prefs` for that category, and fetches their `push_tokens`.
4. Builds a title/body per category and POSTs the batch straight to Expo's push API (`https://exp.host/--/api/v2/push/send`).
5. Runs with the Supabase **service role** key (bypasses RLS entirely, since it needs to read across the whole family) and always returns HTTP 200, even on internal errors, so the originating trigger never sees a failure.

5. Two separate notification mechanisms — don't conflate them

1. **Push notifications** (server-driven, cross-device): the pipeline above. Tells other family members "someone added X." Requires the Edge Function, a registered `push_tokens` row

(`usePushNotifications`), and each recipient's `notification_prefs`.

2. **Local reminders** (device-driven, personal): `useReminderScheduler`, running independently on every device. Reads the current user's own `todos` (ones with `reminder_enabled`, assigned to or created by them) and `events` (ones with `reminder_offsets_minutes`, applicable to them) and schedules native on-device notifications via `expo-notifications`, entirely client-side — no server involvement. Recurring todo reminders are scheduled as native daily/weekly triggers; one-off event reminders are computed per-occurrence (expanding recurrence up to 60 days out) and capped at ~55 pending notifications to stay under the OS's scheduling budget, trimmed soonest-first.

6. Data flow example — adding a to-do item

1. User fills the "New task" modal on the To-Do tab and taps Add.
2. `useTodos().addTodo(...)` runs
`supabase.from("todos").insert({...}).select().single()`.
3. Postgres runs the insert; RLS's WITH CHECK confirms `family_id = get_my_family_id()`; a `replica identity full` row image is recorded for logical replication; the `notify_on_todo_insert` trigger fires `notify_activity()`, which fires-and-forgets a `pg_net` POST to the Edge Function.
4. Back on the client, the returned row is pushed straight into local `todos` state — the UI updates immediately, without waiting on Realtime.
5. Independently, Supabase Realtime broadcasts the INSERT to every other subscribed device's `todos:<familyId>:<instanceId>` channel; each one calls `refetch()` and gets the same new row.
6. Meanwhile, the Edge Function (triggered in step 3) resolves the family's other members, checks each one's `notification_prefs.todos/urgentTodos`, and sends a push notification to their registered device tokens.
7. On every device, `useReminderScheduler` re-runs (its `todos` dependency changed) and reconciles this device's own local on-device reminder schedule if the new todo has a personal reminder set and is assigned to that device's user.

7. Localization

`i18next`/`react-i18next`, with all strings in `lib/i18n/en.ts` / `lib/i18n/fr.ts` as nested namespaced objects (`common.*`, `home.*`, `calendar.*`, `todo.*`, `groceries.*`, `meals.*`, `settings.*`, `setup.*`, `timetable.*`). Initial language is detected from the device locale (`expo-localization`) on first launch, then persisted in `AsyncStorage` once the user picks one explicitly in

Settings — independent of device locale from then on. `app/_layout.tsx` `await`s `i18n` initialization before rendering anything, to avoid a flash of untranslated text.

Because the in-app language can differ from the device locale, all date/time display goes through `lib/dateUtils.ts`'s `formatDate`/`formatTime` helpers (which read `i18n.language` directly, not the device's `Intl` default) rather than calling `Date.prototype.toLocaleDateString()` directly — this was a real bug fixed in this codebase's history (month/day names silently followed the device, not the in-app toggle). French public/school holiday names (`lib/frenchHolidays.ts`) are intentionally **not** translated — they're the actual French names (e.g. "Toussaint," "Ascension") regardless of the app's display language, since that's what they're actually called.

8. Build & Distribution

- **Android:** EAS Build, `preview` profile (`eas.json`) → internal-distribution `.apk`, built via `eas build --platform android --profile preview`. Supabase URL/anon key are injected as build-time env vars from `eas.json`'s `build.preview.env`.
- **Firestore/push config:** `google-services.json` is present in the repo working tree but deliberately **not committed to git** and **not gitignored either** — EAS Build's upload step respects `.gitignore`, so gitignoring it silently breaks Android push notification builds (this happened once and was fixed).
- **iOS:** not yet distributed. EAS Build's managed iOS credentials flow was confirmed (via real build attempts) to require a **paid** Apple Developer Program membership — a free Apple ID isn't sufficient, because Apple's Developer Portal API only exposes a signing "team" for paid accounts. Local Xcode-based signing on a Mac (free, but with a 7-day cert expiry) or AltStore/SideStore sideloading are the free alternatives under consideration; see project memory/prior conversation for the full comparison.

9. Notable conventions & gotchas worth knowing

- **Realtime channel names must be unique per component instance** (``_${table}:${familyId}:${useId()}``) — reusing a literal channel name across two mounted hook instances crashes natively, invisible to the JS error boundary.
- `replica identity full` is set on every realtime-synced table — required for filtered DELETE events to reach other clients at all.
- **Optimistic updates are the norm, not the exception** — every hook mutation either updates local state immediately (with rollback-via-refetch on error) or fetches the inserted row back via `.select().single()`. A hook lacking this (relying solely on the Realtime round-trip) is a known bug pattern that's been hit and fixed multiple times in this codebase.

- `ErrorBoundary`/`ErrorScreen` (the top-level crash fallback) is intentionally **not** localized — it can be hit before or during i18n initialization itself, so it stays hardcoded in English as a safety fallback rather than risking a secondary crash.
- **The anon key is intentionally embedded** in `eas.json` and in the `notify_activity()` trigger SQL — this is expected for a Supabase anon key (it's designed to be public-safe, since every table it can touch is RLS-protected), not an accidental secret leak.
- **Managed ("phone-less") members** are a first-class but structurally different kind of `profiles` row — no `auth.users` row, added/renamed/removed by any/admin family member rather than by themselves.